ARUK & LYCON

Consolidated Project Assessment

Repository-based overview of purpose, architecture, capabilities, risks, and recommended next steps

Prepared: 28 October 2026

Scope: Documentation, selected source files, and GitHub history available during this review. This is not a fresh full test run or independent end-to-end security audit.

Executive summary

Aruk and Lycon are distinct projects with different responsibilities. Aruk focuses on credentials, secrets, access control, and protected passage. Lycon focuses on privacy-first browsing, browser state and permissions, and optional, user-controlled intelligence connections. Their shared ecosystem context does not make their responsibilities interchangeable.

1. Executive Summary

Aruk is a security and credential-management application. Its repository documents a vault, provider-account routing, a Keeper access-control layer, passage logging, APIs, SDK, CLI, and web/desktop/mobile surfaces.

Lycon is a privacy-first browser intended to remain useful without any connected intelligence provider. Its repository documents browser functionality, privacy protections, local-first data, native desktop and Android implementations, and optional intelligence connectors governed by explicit context-sharing rules.

Both repositories contain substantial implementation work. The main question for each is no longer only whether features exist, but whether the documented security and privacy guarantees are consistently enforced across all interfaces and deployment modes.

This report separates repository-backed implementation descriptions from design intent and from historical audit findings that need to be rechecked against current code.

2. Project Identity and Repositories

Project	Primary purpose	Repository
Aruk	Credential and secret management, access policy enforcement, secure routing and passage records.	github.com/Daemon22/aruk
Lycon	Privacy-first browsing with local-first data and optional user-controlled intelligence connections.	github.com/Daemon22/Lycon

3. Aruk — Purpose and Capabilities

3.1 Core capabilities

- **Credential and secret vault:** manages API keys, passwords, OAuth tokens, SSH keys, certificates, service-account credentials and cloud credentials. The README specifies AES-256-GCM encryption at rest and policy-gated access to plaintext credentials.
- **Provider account selection and routing:** tracks provider accounts and usage information, with documented strategies including best, fastest, cheapest, highest quality, round robin and load balancing, plus failover behavior.
- **Keeper and access control:** selected source includes daemon registration, ownership and lineage, access policies, permission scopes, trust levels, capability passes, pass revocation, and access-related audit events.
- **Perimeter and protected passage:** includes perimeter rules, cloud-account operations, and a PassageLog for recording inbound/outbound passage metadata. The audit describes metadata logging rather than storing transferred payloads.
- **Agent integration:** provides a TypeScript SDK, REST API, CLI and agent plugin so other applications can request credentials without embedding provider keys directly in their own code.

3.2 Architecture and platforms

Layer	Implementation described in repository
Core engine	src/lib/api-bank/, Keeper and SecretVault services
Web and REST	Next.js App Router, React and API routes
Persistence	Prisma and SQLite in the audited configuration
SDK / CLI	TypeScript HTTP client and command-line scripts
Desktop	Tauri v2 with Rust
Android	Kotlin, Jetpack Compose and Room
Deployment	Docker, Docker Compose and Caddy

3.3 Security model

- Credentials encrypted at rest.
- Authentication and session expiry.
- User-scoped access to accounts, secrets, policies and audit information.
- Rejection of conflicting cookie and bearer identities.
- Policy-gated credential access.
- Revocation of active passes when a daemon is revoked.
- Append-only PassageLog behavior at the API contract.

These are implementation and documentation claims, not proof that every endpoint or deployment configuration has been independently security-tested.

3.4 Recent repository history

GitHub history retrieved for this review included the following Aruk commits dated 28 October 2026:

- Merge restoring scoped security boundaries and resolving conflicts.
- Keeper’s unified body and completion contract.
- Application shell, canonical art, and truthful backend states.

The commits indicate work on the security boundary, Keeper contract and application interface. Recent local development context reported that the codex/fix-release-blockers merge was completed and pushed and the working tree was clean at that point. Bun was unavailable in that local environment, so type checking, tests and a production build were not verified there. Retrieved GitHub status information showed no status checks for the latest merge commit; that does not prove checks passed.

3.5 Aruk risks and production readiness

The production-readiness audit was dated 1 September 2026. Treat it as a historical baseline rather than a definitive assessment of all October changes.

- **Build gates:** type checking and linting had problems, and Next.js configuration suppressed some build-time checks.
- **Testing breadth:** the wired test command covered cryptography, secret bootstrap and a security-hardening script, but the audit found no broad integration suite.
- **Deployment hardening:** explicit secret configuration, rate limiting, network exposure and reverse-proxy configuration needed attention.

- **Operational recovery:** backup/restore, key rotation, migrations and multi-instance behavior needed clearer guarantees.
- **API contracts:** versioning, consistent errors, pagination limits and generated API documentation were incomplete.
- **Platform releases:** Docker, Tauri packaging and Android release signing were not all independently validated.

These findings must be rechecked against current source before being reported as unresolved today.

Assessment: Aruk has a significant security and credential-management foundation. Its next challenge is to prove that protections hold consistently across every exposed interface and supported deployment, rather than simply adding more features.

4. Lycon — Purpose and Capabilities

4.1 Browser features

- Multi-tab browsing, including private tabs.
- URL and search handling, bookmarks, history and downloads.
- HTTPS-Only behavior and Lycon Shields for advertising/tracking protection.
- Find in page, themes, browser settings, local file browsing and local-first data export/import.
- Voice search, including an offline Vosk option.
- Optional intelligence panel and opt-in account-based synchronization.

The README describes a shared React frontend across desktop, Android and web. Native-only capabilities depend on the environment and should not be assumed to exist in the web version.

4.2 Platform architecture

Component	Implementation described
Frontend	React 19, Vite 7 and TypeScript
Backend	Express and tRPC
Database	Drizzle ORM and MySQL in the reviewed architecture
Identity	OAuth-backed sessions
Desktop	Tauri 2 and Rust
Android	Kotlin and GeckoView
Tests	Vitest
Build	Vite frontend plus esbuild server bundle
Supporting libraries	Manya toolkit and Manya OS nervous-system packages

4.3 The no-agent-completeness principle

The Lycon product vision makes a strong distinction between the browser and any intelligence connected to it. Lycon should remain useful when no provider is configured, no local model is running, no cloud assistant is connected, or the user wants ordinary browsing without sharing page context.

When intelligence is connected, the user should explicitly choose the provider, action and context to share. The vision calls for visible disclosure of outbound destination/context, local-first storage, configurable security sensitivity and no silent telemetry or background page inspection.

Core principle: the browser is the primary product; intelligence is an optional capability.

4.4 Optional intelligence connections and context scopes

The agent-agnostic architecture defines a connector model for local, self-hosted, cloud and browser-native providers.

Scope	Intended meaning
none	Send the user's prompt only.
selection	Send explicitly selected text.
page	Send active-page metadata and readable text after confirmation.
tab	Send active-tab metadata and selected or page context.
local-file	Send a file explicitly selected by the user.

The architecture says Lycon must not silently escalate from selected text to sharing the whole page. It describes Hardened, Balanced and Permissive sensitivity postures. The intended outbound lifecycle is: choose connector, preview destination/context, confirm, dispatch, and record the outcome.

These are documented design requirements; the specification alone does not prove that every connector, permission check or privacy invariant is fully enforced on every platform.

4.5 Lycon Core and canonical state

The backend architecture proposal identifies Lycon Core Constitution v0.3 as the governing architectural authority. The intended rule is that Core owns canonical browser state, identity, effective policy, capabilities, privacy/exposure state, events and state transitions. Other components should use approved interfaces rather than becoming independent authorities.

- The UI should not be the final authority on whether a protected action is allowed.
- Connecting an intelligence should not grant it additional permissions.
- Observing an event should not confer authority to mutate browser state.
- Recording an action afterward is not a substitute for enforcing permission before the action.
- Private browsing needs structural restrictions, not merely a visible private-mode indicator.

These principles are documented; their complete enforcement remains a separate engineering question.

4.6 Backend and synchronization risks

The 24 October 2026 backend proposal identified the following issues in the reviewed baseline:

- **Synchronization validation:** basic revision and payload-size constraints existed, but synchronized JSON was not fully schema-validated.
- **Concurrent writes:** revision read and write were separate operations, allowing concurrent writers to pass the same revision check.
- **Conflict handling:** collection merging by ID lacked deletion tombstones and per-field versions.
- **Backups:** user-triggered JSON export was plaintext, not an encrypted, cryptographically verified archive.
- **Storage authorization:** the existing storage flow did not establish the per-user ownership and authorization guarantees needed for future backup objects.
- **Encryption:** OAuth credentials and authenticated sessions did not establish end-to-end encryption for synchronization.

- **Craft Engine restrictions:** a strategy-restriction fallback appeared not to select fallback data as intended; existing tests did not establish enforcement.
- **Event integration:** the Lycon Manya adapter was not wired into the Express request path, and its fallback could swallow subscriber errors.

These are findings from a proposal reviewing a particular revision, not a claim that all issues remain unfixed today. The proposal reported 161 passing tests, a passing TypeScript check and a passing build for that baseline, with existing build warnings. Those are historical reported results, not a fresh test run in this review.

4.7 Recent Lycon history and release discipline

- Backend architecture proposal — 24 October 2026.
- Canonical-state policy — 24 October 2026.
- Integration/handoff merge — 24 October 2026.

The canonical-state policy says main is the single source of truth, forbids force-pushing or rebasing main, and identifies stable/2026-10-24-green as a rollback point. It specifies a release gate before merging: install from the frozen lockfile, run type checking, tests and build. A green historical checkpoint is useful, but the current branch still needs to pass the gate after subsequent changes.

5. How Aruk and Lycon Fit Together

The cleanest relationship is separation of authority. Aruk protects and manages credentials and access. Lycon controls the browsing environment, local browser state and user consent for information exposure. They can integrate through explicit, authenticated and versioned contracts without merging responsibilities.

Aruk responsibility	Lycon responsibility
Credential and secret access	Browsing and local browser data
Access policies, capability passes and passage records	Browser permissions, privacy posture and exposure controls
Credential security boundary	Browser and user-consent boundary

Example: Lycon could request an authorized credential or capability through Aruk rather than storing its own copy of the secret. Aruk would enforce its access rules; Lycon would retain responsibility for the browser action and the user's consent to share context. This is an architectural example, not proof that the full integration already exists.

The Lycon backend proposal mentions UPMP as a proposed integration for cross-process execution integrity and operational assurance. It does not establish that universal process assurance is already enforced throughout Lycon. The proposal distinguishes Daemon22/UPMP from the separate Manya UPMP-ADT activity-tracking adapter; these should not be treated as the same implementation.

6. Side-by-Side Assessment

Dimension	Aruk	Lycon
Primary purpose	Credential management and security enforcement	Privacy-first browsing
Security boundary	Secrets, access policies, passes and protected passage	Browser state, permissions, session isolation and information exposure
User-facing application	Web dashboard, desktop, Android and CLI	Browser across desktop, Android and web
Main technical challenge	Proving consistent enforcement and safe production deployment	Proving Core invariants across execution surfaces
Testing evidence	September audit reported narrow security tests; October validation needs confirmation	24 October baseline reported 161 passing tests, type check and build
Major concern	Release hardening, contracts and operational recovery	Canonical state, synchronization integrity and privacy guarantees

The test counts come from different dates and test suites. They should not be compared as a measure of which project is better.

7. Recommended Next Steps

7.1 Aruk — security and release verification

- Re-run type checking, linting, tests and production build on the current merged commit.
- Review Keeper, authentication, policy, capability-pass and credential-retrieval paths.
- Test cross-user isolation, credential redaction, authorization failures and revocation.
- Verify environment configuration, backup/restore and deployment hardening.
- Update the September audit against the October implementation.

7.2 Lycon — constitutional enforcement and data integrity

- Read the complete Core Constitution v0.3 and map each invariant to actual enforcement points and tests.
- Audit commands, EngineAdapter, local persistence and native interfaces.
- Resolve concurrent synchronization writes and validate the full sync schema.
- Verify private-session restrictions and explicit intelligence-context authorization.
- Establish reliable backup integrity, recovery and a current passing release gate.

Do not begin by adding another layer of architecture to either project. First establish which guarantees are enforced by current code, which are specified only in documentation, and which are missing.

8. Source Material

Aruk repository: https://github.com/Daemon22/aruk

Aruk production-readiness audit: https://github.com/Daemon22/aruk/blob/main/ARUK_PRODUCTION_READINESS_AUDIT.md

Aruk Keeper implementation: https://github.com/Daemon22/aruk/blob/main/src/lib/keeper/index.ts

Aruk latest reviewed merge: https://github.com/Daemon22/aruk/commit/283515338282c8f6657a6e30ee42c842be55bbe3

Lycon repository: https://github.com/Daemon22/Lycon

Lycon product vision: https://github.com/Daemon22/Lycon/blob/main/LYCON_VISION.md

Lycon agent architecture: https://github.com/Daemon22/Lycon/blob/main/AGENT_ARCHITECTURE.md

Lycon canonical-state policy: https://github.com/Daemon22/Lycon/blob/main/CANONICAL.md

Lycon backend architecture proposal: https://github.com/Daemon22/Lycon/blob/main/docs/BACKEND_ARCHITECTURE_PROPOSAL.md

Lycon Android README: https://github.com/Daemon22/Lycon/blob/main/android/README.md

9. Limitations of This Assessment

This report is based on repository documentation, selected source files and GitHub history retrieved during the review. It does not represent a fresh full test run, complete source-code audit, penetration test, or validation of deployed production environments.

Audit findings from September and October are historical and may have been addressed since the reviewed revision. Verify each against the current branch before treating it as an outstanding defect.

Overall conclusion: Aruk has a substantial credential and access-control foundation whose release readiness needs renewed verification. Lycon has a substantial browser implementation and a clearly articulated privacy/Core-governance architecture, but its own review identifies gaps between constitutional intent and verified enforcement.